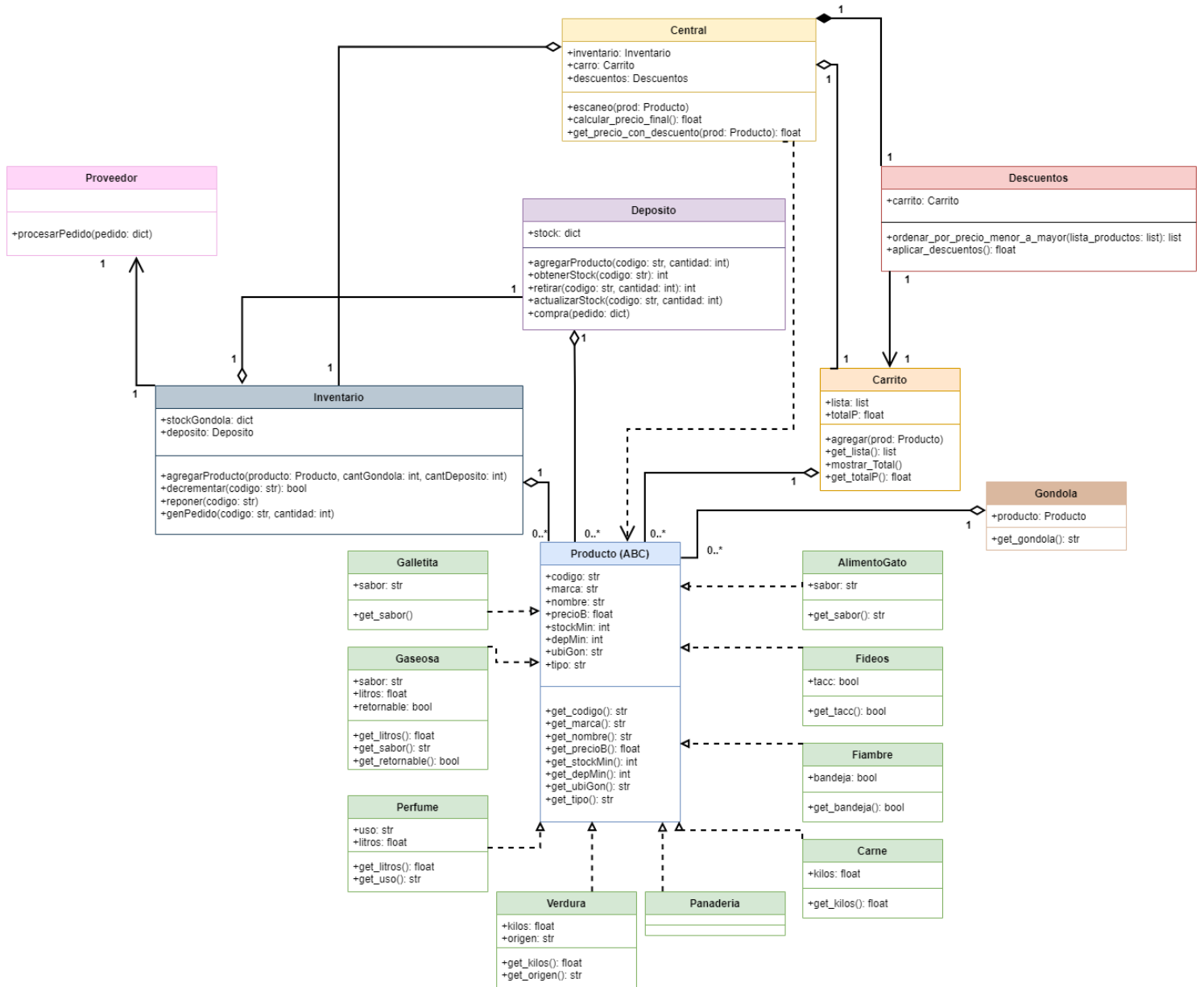




EXPLICACIÓN DEL DIAGRAMA UML

El objetivo de este UML es representar la estructura general del sistema, mostrando las clases utilizadas, sus atributos, sus métodos principales y las relaciones entre ellas. El sistema modela un supermercado donde el cliente puede agregar productos a un carrito, visualizar el total de su compra, acceder a descuentos, y donde además se controla el stock disponible en góndola, depósito y proveedor.

CLASE PRODUCTO

La clase Producto funciona como clase base abstracta para todos los productos del supermercado. Contiene los atributos comunes que comparten las demás clases: código, marca, nombre, precio base, stock mínimo, depósito mínimo, ubicación en góndola y tipo. Estos atributos permiten identificar cada producto, determinar su precio, ubicarlo en el supermercado y controlar su stock.

Sus métodos principales son `get_codigo()`, `get_marca()`, `get_nombre()`, `get_precioB()`, `get_stockMin()`, `get_depMin()`, `get_ubiGon()` y `get_tipo()`. Estos métodos permiten acceder a la información general de cualquier producto.

Producto se relaciona con **Galletita, Gaseosa, Perfume, Verdura, Panaderia, AlimentoGato, Fideos, Fiambre y Carne** mediante una relación de **implementación o realización**. Se utiliza una línea punteada con flecha triangular vacía apuntando hacia Producto porque las clases específicas representan productos concretos que derivan de una clase abstracta. No se coloca cardinalidad en esta relación, ya que no expresa cantidad de objetos.

CLASES DERIVADAS DE PRODUCTO

Las clases Galletita, Gaseosa, Perfume, Verdura, Panaderia, AlimentoGato, Fideos, Fiambre y Carne representan los distintos tipos de productos del supermercado. Todas heredan la estructura general definida en Producto y agregan atributos o métodos propios según las características de cada sector.

Galletita agrega el atributo sabor y el método `get_sabor()`, lo que permite diferenciar las variedades de galletitas.

Gaseosa agrega los atributos sabor, litros y retornable, junto con los métodos `get_litros()`, `get_sabor()` y `get_retornable()`. Esto permite informar el sabor de la bebida, la cantidad de litros y si el envase es retornable.

Perfume agrega los atributos uso y litros, y los métodos `get_litros()` y `get_uso()`. Estos datos permiten diferenciar los productos de perfumería según su uso y cantidad.

Verdura agrega los atributos kilos y origen, y los métodos `get_kilos()` y `get_origen()`. Estos datos permiten representar productos que se venden por peso e indicar su procedencia.

Panaderia se mantiene como una clase específica para representar los productos del sector panadería. En el diagrama no se muestran atributos o métodos propios adicionales, pero permite diferenciar este tipo de productos dentro del sistema.

AlimentoGato agrega el atributo sabor y el método `get_sabor()`, lo que permite diferenciar las variedades del alimento.

Fideos agrega el atributo tacc y el método `get_tacc()`, que indica si el producto contiene o no TACC.

Fiambre agrega el atributo bandeja y el método `get_bandeja()`, que permite indicar si el producto se presenta en bandeja.

Carne agrega el atributo kilos y el método `get_kilos()`, que permite representar el peso del producto.

Todas estas clases se relacionan con **Producto** mediante **implementación o realización**. Esta flecha se utiliza porque cada una representa un tipo concreto de producto a partir de una clase abstracta común. No se coloca cardinalidad, ya que la relación no indica cantidad de objetos, sino especialización.

CLASE CENTRAL

La clase Central representa la controladora centralizada del supermercado. Es la encargada de coordinar el funcionamiento general del sistema, vinculando el carrito, el inventario, los descuentos y los productos.

Sus atributos son inventario, carro y descuentos. El atributo inventario permite acceder al control de stock; carro permite trabajar con el carrito de compras del cliente; y descuentos permite calcular las promociones aplicables.

Sus métodos principales son `escaneo(prod: Producto)`, `calcular_precio_final()` y `get_precio_con_descuento(prod: Producto)`. Estos métodos permiten escanear productos, calcular el total de la compra y obtener el precio final con descuentos.

Central se relaciona con **Inventario** mediante una **agregación**. Se utiliza rombo vacío del lado de Central porque la central tiene asociado un inventario, pero este puede considerarse una clase separada dentro del sistema. La cardinalidad es 1 a 1, ya que una central trabaja con un inventario.

Central se relaciona con **Carrito** también mediante **agregación**. Central utiliza el carrito para registrar la compra, pero el carrito no depende completamente de ella para existir. La cardinalidad es 1 a 1, porque en el funcionamiento del sistema una central trabaja con un carrito.

Central se relaciona con **Descuentos** mediante **composición**. Se utiliza rombo negro del lado de Central porque Descuentos forma parte del funcionamiento interno de la central. La central necesita este objeto para calcular las promociones, por eso se representa una relación más fuerte. La cardinalidad es 1 a 1.

Central se relaciona con **Producto** mediante **dependencia**. Se utiliza una línea punteada con flecha hacia Producto porque Central usa productos de forma puntual en algunos métodos, como `escaneo()` o `get_precio_con_descuento()`, pero no los almacena como parte fija de su estructura. Por este motivo no se coloca cardinalidad.

CLASE CARRITO

La clase Carrito representa el carrito de compras del cliente. Su función es almacenar los productos agregados y calcular el total de la compra.

Sus atributos son lista y totalP. La lista guarda los productos agregados por el cliente, mientras que totalP representa el precio total acumulado.

Sus métodos principales son `agregar(prod: Producto)`, `get_lista()`, `mostrar_Total()` y `get_totalP()`. Estos métodos permiten agregar productos, consultar la lista del carrito, mostrar el total y obtener el precio final.

Carrito se relaciona con **Producto** mediante **agregación**. Se utiliza rombo vacío del lado de Carrito porque el carrito contiene productos, pero los productos pueden existir fuera del carrito, por ejemplo en góndola, inventario o depósito. La cardinalidad es 1 a 0..*, ya que un carrito puede estar vacío o contener muchos productos.

CLASE DESCUENTOS

La clase Descuentos se encarga de calcular y aplicar promociones sobre los productos del carrito.

Su atributo principal es carrito, ya que necesita acceder a los productos agregados por el cliente para poder calcular los descuentos.

Sus métodos principales son `ordenar_por_precio_menor_a_mayor(lista_productos: list)` y `aplicar_descuentos()`. Estos métodos permiten ordenar productos cuando es necesario y calcular el precio final con promociones.

Descuentos se relaciona con **Carrito** mediante una **asociación** dirigida. Se utiliza una línea continua con flecha hacia Carrito porque Descuentos conoce y usa al carrito para obtener la lista de productos. No se usa agregación ni composición porque Descuentos no contiene al carrito como una parte propia, sino que lo consulta para realizar cálculos. La cardinalidad es 1 a 1, porque un objeto Descuentos trabaja sobre un carrito específico.

CLASE INVENTARIO

La clase Inventario se encarga de controlar el stock de los productos disponibles en góndola y de gestionar la reposición cuando corresponde.

Sus atributos son `stockGondola` y `deposito`. `stockGondola` almacena los productos disponibles para la venta y sus cantidades, mientras que `deposito` permite acceder al stock de reserva.

Sus métodos principales son `agregarProducto(producto: Producto, cantGondola: int, cantDeposito: int)`, `decrementar(codigo: str)`, `reponer(codigo: str)` y `genPedido(codigo: str, cantidad: int)`. Estos métodos permiten agregar productos al inventario, disminuir stock cuando se realiza una compra, reponer productos desde depósito y generar pedidos al proveedor.

Inventario se relaciona con **Deposito** mediante **agregación**. Se utiliza rombo vacío del lado de Inventario porque el inventario tiene asociado un depósito, pero el depósito también se representa como una clase independiente. La cardinalidad es 1 a 1, porque un inventario trabaja con un depósito.

Inventario se relaciona con **Proveedor** mediante **asociación** dirigida. Se utiliza una línea continua con flecha hacia Proveedor porque Inventario se comunica con él para gestionar pedidos de reposición. No se usa composición porque el proveedor no forma parte interna del inventario, sino que es una clase externa que se consulta cuando falta stock. La cardinalidad es 1 a 1.

Inventario se relaciona con **Producto** mediante **agregación**. Se utiliza rombo vacío del lado de Inventario porque el inventario administra productos y sus cantidades, pero los productos pueden existir en otras partes del sistema. La cardinalidad es 1 a 0..*, porque un inventario puede registrar cero, uno o muchos productos.

CLASE DEPOSITO

La clase Deposito representa el stock de reserva del supermercado. Contiene los productos que no están directamente disponibles en góndola, pero que pueden usarse para reponerla.

Su atributo principal es stock, que guarda las cantidades disponibles en depósito.

Sus métodos principales son agregarProducto(codigo: str, cantidad: int), obtenerStock(codigo: str), retirar(codigo: str, cantidad: int), actualizarStock(codigo: str, cantidad: int) y compra(pedido: dict). Estos métodos permiten agregar stock, consultar cantidades disponibles, retirar unidades, actualizar el stock y registrar una reposición.

Deposito se relaciona con **Producto** mediante **agregación**. Se utiliza rombo vacío del lado de Deposito porque el depósito almacena productos, pero estos no dependen totalmente del depósito para existir. La cardinalidad es 1 a 0..*, porque un depósito puede contener muchos productos o estar vacío.

CLASE PROVEEDOR

La clase Proveedor representa al proveedor externo encargado de procesar los pedidos de reposición.

En el diagrama no se indican atributos propios para esta clase.

Su método principal es procesarPedido(pedido: dict), que recibe un pedido generado por Inventario y lo procesa.

Proveedor se relaciona con **Inventario** a través de una **asociación** dirigida desde Inventario hacia Proveedor. La flecha indica que Inventario conoce y utiliza al Proveedor para pedir reposición de productos. La cardinalidad es 1 a 1.

CLASE GONDOLA

La clase Gondola representa el lugar físico donde se ubican los productos disponibles para la compra del cliente.

Su atributo principal es producto, que permite asociar la góndola con los productos que contiene.

Su método principal es get_gondola(), que permite obtener la ubicación o información de la góndola.

Gondola se relaciona con **Producto** mediante **agregación**. Se utiliza rombo vacío del lado de Gondola porque una góndola contiene productos, pero estos también pueden existir en otras partes del sistema, como inventario, depósito o carrito. La cardinalidad es 1 a 0..*, porque una góndola puede contener cero, uno o muchos productos.